Introduction

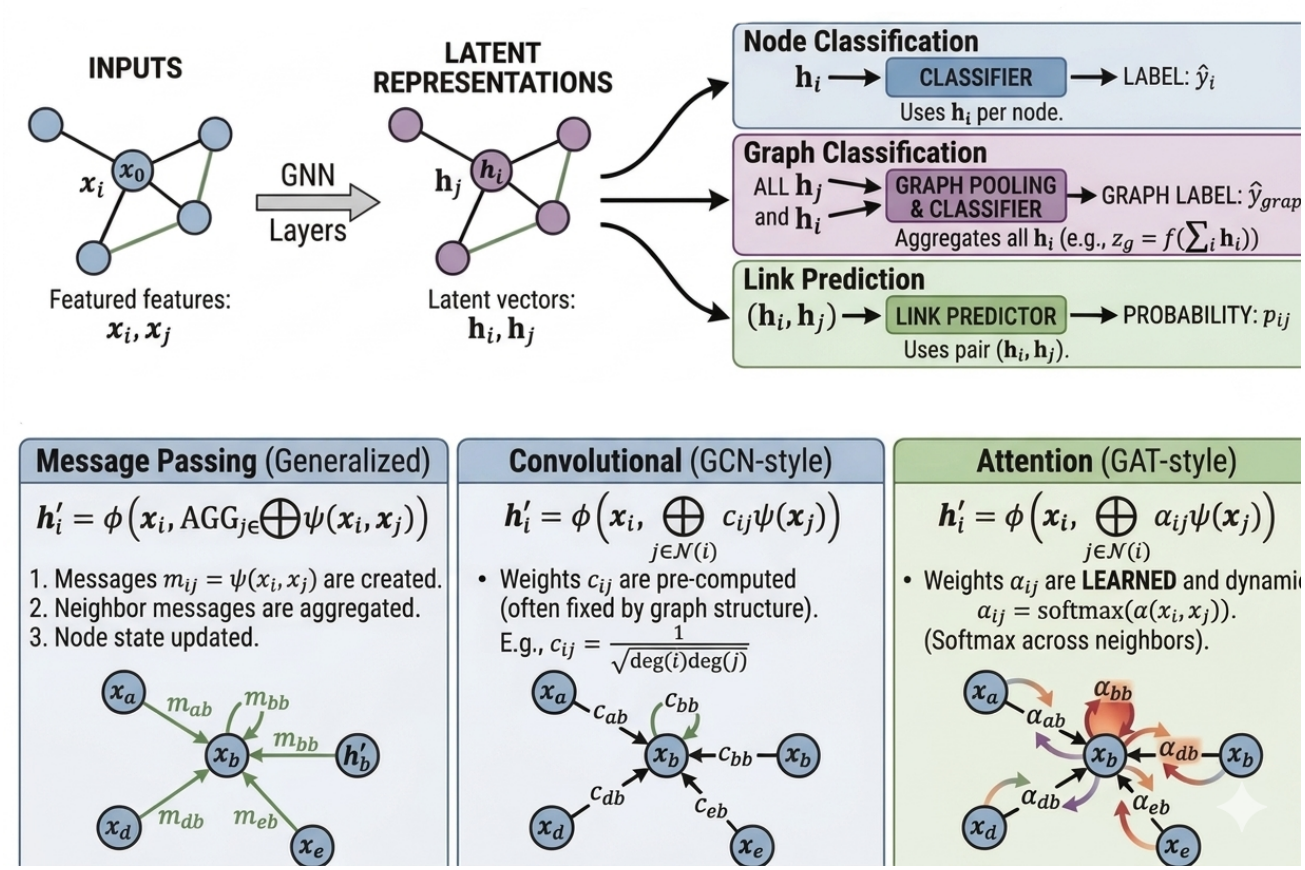

Figure 1. (Inspired by Petar Veličković https://petar-v.com/talks/GNN-Wednesday.pdf)

1. Message Passing in Graph Neural Networks. The fundamental operation in Graph Neural Networks (GNNs) is the **Message Passing** mechanism, which allows a node to update its state by aggregating information from its local neighborhood. The general form is expressed as:

$$\mathbf{h}_i = \phi\left(\mathbf{x}_i, \bigoplus_{j\in\mathcal{N}_i} \psi(\mathbf{x}_i, \mathbf{x}_j)\right) \tag{1}$$

Where:

- $\mathbf{h}_i$ is the updated hidden representation of node i.
- $\mathbf{x}_i$ represents the current features of node i.
- $\mathcal{N}_i$ denotes the set of neighbors for node i.
- ψ is the *Message* function that computes information from neighbor j.
- $\bigoplus$ is a permutation-invariant *Aggregation* operator (e.g., sum, mean, or max).
- ϕ is the *Update* function that combines the node's own features with the aggregated neighborhood message.

2. Graph Convolutional Networks (GCN). In a GCN, the influence of a neighbor is determined by the graph's structure rather than the node's content. A fixed normalization constant c_{ij} is used:

$$\mathbf{h}_i = \phi\left(\mathbf{x}_i, \bigoplus_{j \in \mathcal{N}_i} c_{ij}\psi(\mathbf{x}_j)\right) \tag{2}$$

Here, c_{ij} (often derived from node degrees as $1/\sqrt{d_i d_j}$) scales the messages to account for varying node degrees, ensuring that highly connected "hub" nodes do not overwhelm the representation.

- How it works: In a standard GCN, c_{ij} is usually calculated based on the degrees (number of connections) of nodes i and j. A common choice is $1/\sqrt{d_i d_j}$.
- The Logic: It "scales down" messages from highly connected nodes (hubs). If a neighbor has 1,000 friends, its message to you is treated as less "unique" than a message from a neighbor who only talks to you.
- Key Characteristic: The weight of a neighbor is determined by the graph structure alone. It doesn't care what the actual data ($\mathbf{x}_j$) is.

3. Graph Attention Networks (GAT). GAT introduces a dynamic weighting mechanism. Instead of a fixed constant, it uses an *Attention Coefficient* $a(\mathbf{x}_i, \mathbf{x}_j)$:

$$\mathbf{h}_i = \phi\left(\mathbf{x}_i, \bigoplus_{j \in \mathcal{N}_i} a(\mathbf{x}_i, \mathbf{x}_j)\psi(\mathbf{x}_j)\right) \tag{3}$$

The attention mechanism $a(\cdot)$ allows the model to learn which neighbors are most relevant to node i based on their actual feature values, providing a much higher degree of model flexibility.

- How it works: Instead of using a fixed number based on the graph's shape, the network learns which neighbors are important. It compares the features of node i with neighbor j to decide how much to "listen" to them.
- The Logic: Imagine a social network. Even if you have 100 friends (GCN would weight them similarly), you might only care about the opinions of 2 friends who share your hobbies. GAT allows the model to focus on those 2 relevant neighbors.
- Key Characteristic: The weighting is dynamic. It depends on the node features, meaning the importance of a connection can change depending on the data.

Iterative Message Passing Framework. The state of a graph at any layer t is governed by three primary operations: initialization, aggregation, and updating.

1. Initialization. At the input layer ($t = 0$), each node is initialized with its own feature vector:

$$h_v^{(0)} = x_v, \quad \forall v \in V \tag{4}$$

2. Aggregation (Message Generation). At each iteration t, node v collects messages from its neighbors $u \in \mathcal{N}(v)$. This step may also incorporate edge weights $w(u, v)$:

$$m_v^{(t)} = f_{\text{Agg}}^{(t)} \left(h_v^{(t-1)}, \{\{h_u^{(t-1)}, w(u, v) \mid u \in \mathcal{N}(v)\}\} \right) \tag{5}$$

The multiset notation $\{\{\dots\}\}$ indicates that the aggregation is permutation-invariant.

3. Node Update. The node then computes its new hidden state by combining its previous state with the incoming message:

$$h_v^{(t)} = f_{\text{Up}}(h_v^{(t-1)}, m_v^{(t)}) \tag{6}$$

By repeating this for T steps, a node's final embedding $h_v^{(T)}$ captures information from its T-hop neighborhood.